

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»
Тема: «Архитектура и программирование сопроцессора. Использование
целочисленных команд. Использование вещественных команд»

Выполнил:
Студент 4 курса
Группы АС-66
Рогожин А.С.
Проверила:
Кулик А.Д.

Цель работы: изучение архитектуры и программной модели сопроцессора; практическая работа с системой целочисленных команд

Вариант 20

Постановка задачи раздела 1. Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и целочисленные команды согласно варианту, выданному преподавателем (таблица 2.2). Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль. Программу протестировать по всем условиям.

Задание раздела 1:

Вариант 2

$$y = \begin{cases} 4xy + 5, & x + y > 10; \\ \frac{3xy + 2y}{x^2 + y^2 + 1}, & x + y < -2; \\ 6x - 2y^2 + 1, & -2 \leq x + y \leq 1. \end{cases}$$

(10 шакалов из 10)

Код программы

```
.386
.model flat, stdcall
option casemap:none
includelib kernel32.lib

ExitProcess PROTO :DWORD
GetStdHandle PROTO :DWORD
ReadConsoleA PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD
WriteConsoleA PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD
SetConsoleCP PROTO :DWORD
SetConsoleOutputCP PROTO :DWORD

STD_INPUT_HANDLE EQU -10
STD_OUTPUT_HANDLE EQU -11
MAX_LEN EQU 32

.data
promptX db "Vvedite x: ", 0
promptY db "Vvedite y: ", 0
branchMsg db 13, 10, "Vibrana vetka i formula: ", 0
b1Text db "1 (x + y > 10) ----> y = 4xy + 5", 0
b2Text db "2 (x + y < -2) ----> y = (3xy + 2y) / (x^2 + y^2 + 1)", 0
b3Text db "3 (-2 <= x+y <= 10) ----> y = 6x - 2y^2 + 1", 0
```

```

resMsg      db 13, 10, "Rezultat (int): ", 0
newline     db 13, 10, 0
bufX        db MAX_LEN dup(0)
bufY        db MAX_LEN dup(0)
charsRead   dd 0
valX        dd 0
valRes      dd 0
outBuf      db 32 dup(0)
valFour     dd 4
valFive     dd 5
valThree    dd 3
valTwo      dd 2
valSix      dd 6
valOne      dd 1

```

.code

PrintString PROC

```

    push eax
    push ecx
    push edx
    push esi
    push edi
    mov esi, edx
    xor ecx, ecx

```

PS_Loop:

```

    cmp byte ptr [esi], 0
    je PS_Done
    inc esi
    inc ecx
    jmp PS_Loop

```

PS_Done:

```

    push STDOUT_HANDLE
    call GetStdHandle
    mov edi, eax
    sub esp, 4
    mov eax, esp
    push 0
    push eax
    push ecx
    push edx
    push edi
    call WriteConsoleA
    add esp, 4
    pop edi

```

```
    pop esi
    pop edx
    pop ecx
    pop eax
    ret
```

PrintString ENDP

ParseInt PROC

```
    push ebx
    push ecx
    push edx
    push esi
    mov esi, edx
    xor eax, eax
    xor ebx, ebx
    mov dl, byte ptr [esi]
    cmp dl, '-'
    jne PI_CheckDigit
    mov ebx, 1
    inc esi
    jmp PI_Loop
```

PI_CheckDigit:

```
    cmp dl, '+'
    jne PI_Loop
    inc esi
```

PI_Loop:

```
    mov dl, byte ptr [esi]
    cmp dl, '0'
    jb PI_Done
    cmp dl, '9'
    ja PI_Done
    sub dl, '0'
    imul eax, eax, 10
    movzx edx, dl
    add eax, edx
    inc esi
    jmp PI_Loop
```

PI_Done:

```
    cmp ebx, 1
    jne PI_Exit
    neg eax
```

PI_Exit:

```
    pop esi
    pop edx
```

```
pop ecx
```

```
pop ebx
```

```
ret
```

```
ParseInt ENDP
```

```
IntToString PROC
```

```
push ebx
```

```
push ecx
```

```
push edx
```

```
push esi
```

```
push edi
```

```
mov edi, edx
```

```
mov ecx, eax
```

```
test ecx, ecx
```

```
jns ITS_Positive
```

```
mov byte ptr [edi], '-'
```

```
inc edi
```

```
neg ecx
```

```
ITS_Positive:
```

```
mov eax, ecx
```

```
mov ebx, 10
```

```
xor ecx, ecx
```

```
ITS_PushLoop:
```

```
xor edx, edx
```

```
div ebx
```

```
push edx
```

```
inc ecx
```

```
test eax, eax
```

```
jnz ITS_PushLoop
```

```
ITS_PopLoop:
```

```
pop edx
```

```
add dl, '0'
```

```
mov byte ptr [edi], dl
```

```
inc edi
```

```
dec ecx
```

```
jnz ITS_PopLoop
```

```
mov byte ptr [edi], 0
```

```
pop edi
```

```
pop esi
pop edx
pop ecx
pop ebx
ret
```

IntToString ENDP

ReadLine PROC

```
push ebx
push ecx
push edx
push esi
push edi
```

```
mov esi, edx
push STD_INPUT_HANDLE
call GetStdHandle
mov edi, eax
```

```
push 0
push OFFSET charsRead
push MAX_LEN
push esi
push edi
call ReadConsoleA
```

```
mov ecx, charsRead
sub ecx, 2
```

```
cmp ecx, 0
jl RL_Empty
```

```
mov byte ptr [esi + ecx], 0
jmp RL_Exit
```

RL_Empty:

```
mov byte ptr [esi], 0
```

RL_Exit:

```
pop edi
pop esi
pop edx
pop ecx
```

```
    pop ebx
    ret
ReadLine ENDP
```

```
Branch1 PROC
    push eax
    push edx

    fild dword ptr [esp + 4]
    fild dword ptr [esp]
    fmulp
    fimul dword ptr [valFour]
    fiadd dword ptr [valFive]
    fistp dword ptr [valRes]

    add esp, 8
    ret
Branch1 ENDP
```

```
Branch2 PROC
    push eax
    push edx

    fild dword ptr [esp + 4]
    fild dword ptr [esp]
    fmulp
    fimul dword ptr [valThree]

    fild dword ptr [esp]
    fimul dword ptr [valTwo]
    faddp

    fild dword ptr [esp + 4]
    fild dword ptr [esp + 4]
    fmulp

    fild dword ptr [esp]
    fild dword ptr [esp]
    fmulp

    faddp
    fiadd dword ptr [valOne]
```

```
    fdivp
    fistp dword ptr [valRes]
```

```
    add esp, 8
    ret
```

Branch2 ENDP

Branch3 PROC

```
    push eax
    push edx
```

```
    fild dword ptr [esp + 4]
    fimul dword ptr [valSix]
```

```
    fild dword ptr [esp]
    fild dword ptr [esp]
    fmulp
    fimul dword ptr [valTwo]
```

```
    fsubp
    fiadd dword ptr [valOne]
    fistp dword ptr [valRes]
```

```
    add esp, 8
    ret
```

Branch3 ENDP

start PROC

```
    push 1251
    call SetConsoleCP
    push 1251
    call SetConsoleOutputCP
```

```
    mov edx, OFFSET promptX
    call PrintString
    mov edx, OFFSET bufX
    call ReadLine
    mov edx, OFFSET bufX
    call ParseInt
    mov valX, eax
```

```
    mov edx, OFFSET promptY
    call PrintString
```

```
mov edx, OFFSET bufY
call ReadLine
mov edx, OFFSET bufY
call ParseInt
mov edx, eax
```

```
mov eax, valX
```

```
push eax
push edx
add eax, edx
```

```
cmp eax, 10
jg DoBranch1
```

```
cmp eax, -2
jl DoBranch2
```

```
mov edx, OFFSET branchMsg
call PrintString
mov edx, OFFSET b3Text
call PrintString
```

```
pop edx
pop eax
call Branch3
jmp ShowResult
```

DoBranch1:

```
mov edx, OFFSET branchMsg
call PrintString
mov edx, OFFSET b1Text
call PrintString
```

```
pop edx
pop eax
call Branch1
jmp ShowResult
```

DoBranch2:

```
mov edx, OFFSET branchMsg
call PrintString
mov edx, OFFSET b2Text
```

```
call PrintString
```

```
pop edx
```

```
pop eax
```

```
call Branch2
```

ShowResult:

```
mov edx, OFFSET resMsg
```

```
call PrintString
```

```
mov eax, valRes
```

```
mov edx, OFFSET outBuf
```

```
call IntToString
```

```
mov edx, OFFSET outBuf
```

```
call PrintString
```

```
mov edx, OFFSET newline
```

```
call PrintString
```

```
push 0
```

```
call ExitProcess
```

```
start ENDP
```

```
END start
```

Результат работы программы:

```
PS D:\Uni\АПО ЭВМИС\АРОЕVMiS_AS66\reports\Rogozhin\lab4\src> .\lab41.exe
Vvedite x: 6
Vvedite y: 7

Vibrana vetka i formula: 1 (x + y > 10) ---> y = 4xy + 5
Rezultat (int): 173
PS D:\Uni\АПО ЭВМИС\АРОЕVMiS_AS66\reports\Rogozhin\lab4\src> 
PS D:\Uni\АПО ЭВМИС\АРОЕVMiS_AS66\reports\Rogozhin\lab4\src> .\lab41.exe
Vvedite x: -3
Vvedite y: -1

Vibrana vetka i formula: 2 (x + y < -2) ---> y = (3xy + 2y) / (x^2 + y^2 + 1)
Rezultat (int): 1
PS D:\Uni\АПО ЭВМИС\АРОЕVMiS_AS66\reports\Rogozhin\lab4\src> +
```

```

PS D:\Uni\АПО ЭВМиС\АПОЕВМиС_AS66\reports\Rogozhin\lab4\src> .\lab41.exe
Vvedite x: -4
Vvedite y: 5

Vibrana vetka i formula: 3 (-2 <= x+y <= 1)      ---> y = 6x - 2y^2 + 1
Rezultat (int): -73
PS D:\Uni\АПО ЭВМиС\АПОЕВМиС_AS66\reports\Rogozhin\lab4\src>

```

Постановка задачи раздела 2. Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и вещественные команды согласно варианту, выданному преподавателем (таблица 2.2). Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль. Программу протестировать по всем условиям

Задание раздела 2: то же самое

Код программы

```

.386
.model flat, stdcall
option casemap:none

includelib kernel32.lib

ExitProcess PROTO :DWORD
GetStdHandle PROTO :DWORD
ReadConsoleA PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD
WriteConsoleA PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD
SetConsoleCP PROTO :DWORD
SetConsoleOutputCP PROTO :DWORD

STD_INPUT_HANDLE EQU -10
STD_OUTPUT_HANDLE EQU -11

MAX_LEN EQU 32

.data

promptX db "Vvedite x (float): ", 0
promptY db "Vvedite y (float): ", 0

branchMsg db 13,10, \
            "Vibrana vetka i formula (float): ", 0

```

```
b1Text db "1 (x + y > 10) ----> y = 4xy + 5", 0
b2Text db "2 (x + y < -2) ----> y = (3xy + 2y) / (x^2 + y^2 + 1)", 0
b3Text db "3 (-2 <= x+y <= 10) ----> y = 6x - 2y^2 + 1", 0
```

```
resMsg db 13,10, \
        "Rezultat (float): ", 0
```

```
newline db 13,10,0
```

```
bufX db MAX_LEN dup(0)
bufY db MAX_LEN dup(0)
```

```
charsRead dd 0
```

```
valX real4 0.0
valY real4 0.0
valRes real4 0.0
```

```
outBuf db 64 dup(0)
```

```
f_four    real4 4.0
f_five    real4 5.0
f_three    real4 3.0
f_two     real4 2.0
f_six     real4 6.0
f_one     real4 1.0
```

```
f_ten     real4 10.0
f_minus2  real4 -2.0
```

```
tempInt dd 0
tempFrac dd 0
tenInt dd 10
```

```
tempScaled dd 0
tempWhole  dd 0
tempRemainder dd 0
hundredInt dd 100
```

```
.code
```

```
PrintString PROC
```

```
push eax
push ecx
push edx
push esi
push edi

mov esi, edx
xor ecx, ecx
```

PS_Loop:

```
cmp byte ptr [esi], 0
je PS_Done

inc esi
inc ecx

jmp PS_Loop
```

PS_Done:

```
push STDOUT_HANDLE
call GetStdHandle

mov edi, eax

sub esp, 4
mov eax, esp

push 0
push eax
push ecx
push edx
push edi

call WriteConsoleA

add esp, 4

pop edi
pop esi
pop edx
pop ecx
```

```
pop eax
```

```
ret
```

```
PrintString ENDP
```

```
ReadLine PROC
```

```
push ebx
```

```
push ecx
```

```
push edx
```

```
push esi
```

```
push edi
```

```
mov esi, edx
```

```
push STD_INPUT_HANDLE
```

```
call GetStdHandle
```

```
mov edi, eax
```

```
push 0
```

```
push OFFSET charsRead
```

```
push MAX_LEN
```

```
push esi
```

```
push edi
```

```
call ReadConsoleA
```

```
test eax, eax
```

```
jz RL_Error
```

```
mov ecx, charsRead
```

```
cmp ecx, 0
```

```
je RL_Empty
```

```
RL_Remove:
```

```
cmp ecx, 0
```

```
je RL_Empty
```

```
mov eax, ecx
```

```
dec eax
```

```
cmp byte ptr [esi + eax], 13
```

```
je RL_Delete
```

```
cmp byte ptr [esi + eax], 10
```

```
je RL_Delete
```

```
jmp RL_Finish
```

RL_Delete:

```
mov byte ptr [esi + eax], 0
```

```
dec ecx
```

```
jmp RL_Remove
```

RL_Finish:

```
mov byte ptr [esi + ecx], 0
```

```
mov eax, ecx
```

```
jmp RL_Exit
```

RL_Empty:

```
mov byte ptr [esi], 0
```

```
xor eax, eax
```

```
jmp RL_Exit
```

RL_Error:

```
mov byte ptr [esi], 0
```

```
xor eax, eax
```

RL_Exit:

```
pop edi
```

```
pop esi
```

```
pop edx
```

```
pop ecx
```

```
pop ebx
```

```
ret
```

```
ReadLine ENDP
```

```
ParseFloat PROC
```

```
push eax
```

```
push ebx
```

```
push ecx
```

```
push edx
```

```
push esi
```

```
push edi
```

```
mov esi, edx
```

```
xor ebx, ebx
```

```
mov al, byte ptr [esi]
```

```
cmp al, '-'
```

```
jne PF_CheckPlus
```

```
mov ebx, 1
```

```
inc esi
```

```
jmp PF_Start
```

```
PF_CheckPlus:
```

```
cmp al, '+'
```

```
jne PF_Start
```

```
inc esi
```

```
PF_Start:
```

```
fldz
```

```
PF_IntegerLoop:
```

```
mov al, byte ptr [esi]
```

```
cmp al, 0
je PF_Done
```

```
cmp al, '.'
je PF_FractionStart
```

```
cmp al, '0'
jb PF_Done
```

```
cmp al, '9'
ja PF_Done
```

```
sub al, '0'
```

```
movzx eax, al
mov tempInt, eax
```

```
fimul tenInt
fiadd tempInt
```

```
inc esi
```

```
jmp PF_IntegerLoop
```

PF_FractionStart:

```
inc esi
```

```
mov tempFrac, 0
mov edi, 10
```

PF_FractionLoop:

```
mov al, byte ptr [esi]
```

```
cmp al, 0
je PF_FractionDone
```

```
cmp al, '0'
jb PF_FractionDone
```

```
cmp al, '9'
```

ja PF_FractionDone

sub al, '0'

movzx eax, al

mov tempInt, eax

mov eax, edi

mov tempFrac, eax

fild tempInt

fidiv tempFrac

faddp st(1), st

imul edi, 10

inc esi

jmp PF_FractionLoop

PF_FractionDone:

PF_Done:

cmp ebx, 0

je PF_Return

fchs

PF_Return:

pop edi

pop esi

pop edx

pop ecx

pop ebx

pop eax

ret

ParseFloat ENDP

IntToString PROC

```
push ebx
push ecx
push edx
push esi
push edi
```

```
mov edi, edx
mov ecx, eax
```

```
test ecx, ecx
jns ITS_Positive
```

```
mov byte ptr [edi], '-'
inc edi
```

```
neg ecx
```

ITS_Positive:

```
cmp ecx, 0
jne ITS_NotZero
```

```
mov byte ptr [edi], '0'
inc edi
```

```
mov byte ptr [edi], 0
```

```
jmp ITS_Done
```

ITS_NotZero:

```
mov eax, ecx
mov ebx, 10
xor ecx, ecx
```

ITS_PushLoop:

```
xor edx, edx
div ebx
```

```
push edx
```

```
inc ecx
```

```
test eax, eax
```

```
jnz ITS_PushLoop
```

```
ITS_PopLoop:
```

```
pop edx
```

```
add dl, '0'
```

```
mov byte ptr [edi], dl
```

```
inc edi
```

```
dec ecx
```

```
jnz ITS_PopLoop
```

```
mov byte ptr [edi], 0
```

```
ITS_Done:
```

```
pop edi
```

```
pop esi
```

```
pop edx
```

```
pop ecx
```

```
pop ebx
```

```
ret
```

```
IntToString ENDP
```

```
FloatToString PROC
```

```
push eax
```

```
push ebx
```

```
push ecx
```

```
push edx
```

```
push esi
```

```
push edi
```

```
mov edi, edx
```

ftst

fnstsw ax

sahf

jnc FTS_Positive

mov byte ptr [edi], '-'

inc edi

fchs

FTS_Positive:

fld st(0)

fimul hundredInt

fistp tempScaled

fstp st(0)

mov eax, tempScaled

xor edx, edx

mov ebx, 100

div ebx

mov tempWhole, eax

mov tempRemainder, edx

mov eax, tempWhole

mov edx, edi

call IntToString

mov esi, edi

FTS_FindEnd:

cmp byte ptr [esi], 0

```
je FTS_WriteDot
```

```
inc esi
```

```
jmp FTS_FindEnd
```

```
FTS_WriteDot:
```

```
mov byte ptr [esi], '.'
```

```
inc esi
```

```
mov eax, tempRemainder
```

```
cmp eax, 10
```

```
jae FTS_WriteFraction
```

```
mov byte ptr [esi], '0'
```

```
inc esi
```

```
FTS_WriteFraction:
```

```
mov edx, esi
```

```
call IntToString
```

```
pop edi
```

```
pop esi
```

```
pop edx
```

```
pop ecx
```

```
pop ebx
```

```
pop eax
```

```
ret
```

```
FloatToString ENDP
```

```
Branch1Float PROC
```

```
fld valX
```

```
fld valY
```

```
fmulp st(1), st
```

```
fmul f_four  
fadd f_five
```

```
fstp valRes
```

```
ret
```

```
Branch1Float ENDP
```

```
Branch2Float PROC
```

```
fld valX  
fld valY
```

```
fmlp st(1), st  
fmul f_three
```

```
fld valY  
fmul f_two
```

```
faddp st(1), st
```

```
fld valX  
fld valX
```

```
fmlp st(1), st
```

```
fld valY  
fld valY
```

```
fmlp st(1), st
```

```
faddp st(1), st  
fadd f_one
```

```
fdivp st(1), st
```

```
fstp valRes
```

```
ret
```

Branch2Float ENDP

Branch3Float PROC

```
fld valX
fmul f_six

fld valY
fld valY

fmulp st(1), st

fmul f_two

fsubp st(1), st

fadd f_one

fstp valRes

ret
```

Branch3Float ENDP

start PROC

```
push 1251
call SetConsoleCP

push 1251
call SetConsoleOutputCP

mov edx, OFFSET promptX
call PrintString

mov edx, OFFSET bufX
call ReadLine

mov edx, OFFSET bufX
call ParseFloat
```

fstp valX

mov edx, OFFSET promptY
call PrintString

mov edx, OFFSET bufY
call ReadLine

mov edx, OFFSET bufY
call ParseFloat

fstp valY

fld valX
fadd valY

fcom f_ten

fnstsw ax
sahf

ja DoBranch1

fcom f_minus2

fnstsw ax
sahf

jb DoBranch2

fstp st(0)

mov edx, OFFSET branchMsg
call PrintString

mov edx, OFFSET b3Text
call PrintString

call Branch3Float

jmp ShowResult

DoBranch1:

```
fstp st(0)
```

```
mov edx, OFFSET branchMsg  
call PrintString
```

```
mov edx, OFFSET b1Text  
call PrintString
```

```
call Branch1Float
```

```
jmp ShowResult
```

DoBranch2:

```
fstp st(0)
```

```
mov edx, OFFSET branchMsg  
call PrintString
```

```
mov edx, OFFSET b2Text  
call PrintString
```

```
call Branch2Float
```

ShowResult:

```
mov edx, OFFSET resMsg  
call PrintString
```

```
fld valRes
```

```
mov edx, OFFSET outBuf
```

```
call FloatToString
```

```
mov edx, OFFSET outBuf  
call PrintString
```

```
mov edx, OFFSET newline  
call PrintString
```

```
push 0
```

```
call ExitProcess
```

```
start ENDP
```

END start **Результат работы программы:**

```
Vvedite x (float): 6.5
Vvedite y (float): 3.8

Vibrana vetka i formula (float): 1 (x + y > 10) ---> y = 4xy + 5
Rezultat (float): 103.80
PS D:\Uni\АПО ЭВМИС\АРОЕVMiS_AS66\reports\Rogozhin\lab4\src>

Vvedite x (float): -2
Vvedite y (float): -3

Vibrana vetka i formula (float): 2 (x + y < -2) ---> y = (3xy + 2y) / (x^2 + y^2 + 1)
Rezultat (float): 0.86
Vvedite x (float): 3.2
Vvedite y (float): 2.3

Vibrana vetka i formula (float): 3 (-2 <= x+y <= 10) ---> y = 6x - 2y^2 + 1
Rezultat (float): 9.62
```

Вывод: в ходе лабораторной работы была изучена архитектура и программная модель математического сопроцессора, а также основные принципы работы с его регистрами и командами. На практике были освоены загрузка и обработка вещественных чисел, выполнение арифметических операций и организация вычислений с использованием стека сопроцессора.